



浙江大學
ZHEJIANG UNIVERSITY

信号与系统

短时傅里叶变换可视化

姓名 _____ XX

学号 _____ XX

学院 _____ 信息与工程学院

2026 年 9 月 19 日

摘要

傅里叶变换能够揭示信号的全局频率组成，但无法回答“某个频率在什么时候出现”这一时变分析问题。针对该局限，本文围绕短时傅里叶变换（Short-Time Fourier Transform, STFT）展开课程设计。首先，通过 `waveform.py` 绘制两路离散信号的时域波形，再利用 `fft_analyze.py` 调用手写数值傅里叶变换验证两者在全局频谱上几乎一致；随后基于 `stft_analyzer.py` 实现的滑动矩形窗逐帧变换，绘制不同窗长下的时频图，分析时间分辨率与频率分辨率之间的权衡；最后以自选音乐为对象，利用 `stft_animator.py` 与 `fft_animator.py` 分别生成滚动时频动画与彩虹雨频谱动画，并将所有分析功能封装为 Flask 后端 API，结合 Vue.js 前端部署为可交互网站。实验结果表明：仅观察频谱图无法区分局部时序结构，而 STFT 可有效揭示两路信号在频率出现顺序上的差异；窗长减小时时频图边界更清晰但频率纹理变粗；对于音乐信号，STFT 能直观反映主旋律、谐波和瞬态打击成分的时间演化。

关键词：短时傅里叶变换；时频分析；矩形窗；音频可视化；Web 部署

1 问题提出与研究意义

经典傅里叶变换通过对整个时间轴进行积分，将时域信号映射到频域，从而揭示信号所包含的频率分量。然而这种变换是全局性的：一旦频率成分随时间发生变化，普通频谱图只能给出“包含了哪些频率”，却无法说明“这些频率分别在什么时刻出现、持续了多久”。对于语音、音乐以及非平稳工程信号，这一缺陷会显著限制分析能力。

本课程设计围绕“短时傅里叶变换可视化”这一主题展开，目标包括四个方面：

1. 对给定的两路离散信号进行时域观察与傅里叶变换，比较其全局频谱特性；
2. 采用矩形窗自行实现 STFT，绘制两路信号在不同窗长下的时频图，讨论时间分辨率与频率分辨率之间的权衡；
3. 选择一段音乐，分别生成滚动时频动画（STFT 动画）和逐帧频谱动画（FFT 动画），实现音频的动态可视化；
4. 将所有分析功能封装为 Web 应用，在线部署，使用户能够上传任意音频文件并实时获取时频分析结果。

该课题将“全局频谱分析”与“局部时频分析”联系起来，有助于理解非平稳信号处理中最核心的时频分辨率权衡问题，也为后续学习语音处理、音乐信息检索和雷达声纳信号分析提供基础。

2 实验原理与实现方法

2.1 连续傅里叶变换与离散实现

连续时间信号 $x(t)$ 的傅里叶变换定义为

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft} dt.$$

题目给出的信号已按采样率 $f_s = 4410$ Hz 离散化，将积分改写为数值求和。若信号长度为 N ，采样时刻为 $t_n = n/f_s$ ，频率采样点为 $f_k = kf_s/N$ ，则：

$$X(f_k) \approx \sum_{n=0}^{N-1} x[n]e^{-j2\pi f_k t_n} \cdot \frac{1}{f_s}.$$

`py_tools/lib/manual_fft.py` 按上式实现了三个函数：`build_time_samples` 生成时间轴 t_n ，`build_frequency_samples` 生成频率轴 f_k ，`fft` 通过双重循环完成数值求和。`fft_analyze.py` 调用上述函数，从 `.mat` 文件中读取信号变量（优先取 `z`，兼容 `zz`）并将幅值谱保存为图像；`waveform.py` 则负责绘制时域波形，两者的输出均保存至 `py_tools/output`。

2.2 短时傅里叶变换

为了获得频率随时间的演化规律，需要用有限长度的窗函数 $w(t)$ 在时间轴上滑动，每移动到一个位置就截取局部片段并进行傅里叶变换。连续形式为

$$\text{STFT}_x(t, f) = \int_{-\infty}^{\infty} x(\tau)w(\tau - t)e^{-j2\pi f\tau} d\tau.$$

离散实现可写成

$$X_m[k] = \sum_{n=0}^{L-1} x[n + mR] w[n] e^{-j2\pi kn/L},$$

其中 L 为窗长， R 为帧移， m 为帧序号。题目要求第二部分使用矩形窗：

$$w[n] = \begin{cases} 1, & 0 \leq n < L, \\ 0, & \text{其他.} \end{cases}$$

STFT 的核心约束在于时间分辨率与频率分辨率不能同时任意提高。窗长为 L 时，频率分辨率近似为

$$\Delta f = \frac{f_s}{L},$$

帧移为 R 时，时间分辨率近似为

$$\Delta t = \frac{R}{f_s}.$$

较小的窗长能够更清楚地定位频率变化发生的时刻，但频率刻画变粗；较大的窗长提高频率分辨率，但会模糊时间上的突变边界。

`stft_analyzer.py` 中的 `stft` 函数对每一帧直接调用 `numpy.fft.fft`，而非复用手写的双重循环。这是因为 STFT 需要对大量重叠帧重复计算频谱，FFT 将单帧计算复杂度降为 $O(L \log L)$ ，远低于双重循环的 $O(L^2)$ ，更适合批量时频图绘制和长音频动画生成。

2.3 代码组织与实验流程

项目代码按功能分为两个部分：`py_tools` 目录存放离线分析脚本，`signal_analyzer` 目录存放 Web 部署代码。

离线分析脚本（`py_tools`）共包含以下模块：

1. `lib/manual_fft.py`: 提供手写数值傅里叶变换函数 `fft`，以及配套的时间轴生成函数 `build_time_samples` 和频率轴生成函数 `build_frequency_samples`；
2. `signal_stats.py`: 计算两路信号的均值、方差和总能量，验证它们在全局统计特性上的一致性；
3. `waveform.py`: 读取 `signal1.mat` 和 `signal2.mat`，通过 `Config` 类统一管理图像尺寸（12 × 4 英寸）和输出路径，绘制并保存时域波形图；
4. `music_waveform.py`: 分析音乐文件的波形图；
5. `fft_analyze.py`: 调用 `manual_fft` 计算两路信号的全局频谱，读取 `.mat` 变量时兼容 `z` 与 `zz` 两种命名，将幅值谱图保存到 `py_tools/output`；
6. `music_fft.py`: 分析音乐信号频谱图
7. `stft_analyzer.py`: 实现带矩形窗的 STFT 核心函数 `stft`，默认帧移为半窗重叠（ $R = L/2$ ），批量输出不同窗长下的时频图；
8. `music_stft.py`: 分析音乐信号的时频图
9. `stft_animator.py`: 读取 `self_songbie.mp3`，复用 `stft_analyzer.py` 的 `stft` 函数生成滚动时频动画，每帧展示宽度为 10 帧的滑动时间窗口，使时频图随音乐连续滚动，由 `FFMpegWriter` 导出静音 MP4 后再调用 `ffmpeg` 混入原始音轨；
10. `fft_animator.py`: 逐帧对音频信号做矩形窗 FFT，将 dB 标度频谱绘制为折线（对数频率轴），同时在背景随机生成 120 条 HSV 彩色线段叠加为“彩虹雨”装饰效果，生成视觉更丰富的频谱动画；
11. `lib/log.py`: 轻量日志模块，同时输出至控制台和 `py_tools/data/project.log`；

12. `py_tools/output`: 统一保存所有频谱图、时频图和视频输出, 便于后续插图整理与实验对比。

Web 应用 (`signal_analyzer`) 采用前后端分离架构: 后端由 Python Flask 提供 REST API, 前端由 Vue.js 实现交互界面, 通过 Docker Compose 统一部署。后端核心模块如下:

1. `services/stft_analyzer.py`: STFT 引擎, 在离线版矩形窗基础上增加了 Hann、Hamming、Blackman、Bartlett、Kaiser 五种窗函数, 并支持独立设置 FFT 点数 (零填充), 接口签名与离线版保持一致;
2. `services/signal_processing.py`: 接收 `AnalysisParams` 字典 (含 `sr`、`n_fft`、`hop_length`、`window` 等字段), 返回波形图、频谱图和时频图的 Base64 编码字符串;
3. `services/stft_animation.py`: 接收 `AnimationParams` 字典, 生成滚动时频动画 MP4;
4. `api/signal.py`: 定义 `POST /upload`、`GET /analyze/<filename>`、`POST /animate/<filename>` 等 REST 接口, 从查询字符串或请求体中读取可调参数, 动画渲染通过异步任务队列完成, 避免长时间阻塞请求。

`materials` 文件夹内部存放分析的数据和分析出的结果。

`signal1.mat` 原始信号 1

`signal2.mat` 原始信号 2

`self_songbie.mp3` 时频动画的原始音乐文件

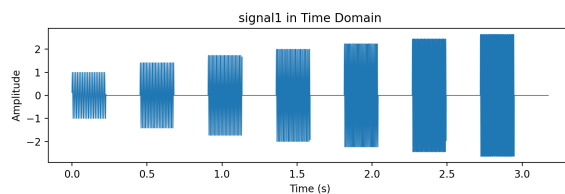
`music_stft.mp4` 音乐的滚动时频动画

`music_fft.mp4` 音乐的彩虹雨频谱动画

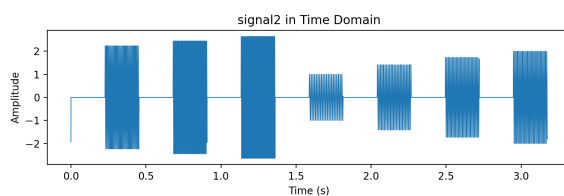
据此, 全文实验流程可概括为三个阶段: 首先, 运行 `waveform.py` 和 `fft_analyze.py` 获得两路给定信号的时域波形与全局频谱, 建立“频率成分基本一致”的基线结论; 其次, 运行 `stft_analyzer.py`, 在多组窗长条件下比较两路信号的时频图, 分析分辨率折中关系; 最后, 运行 `stft_animator.py` 和 `fft_animator.py` 对音乐片段生成两种形式的动画, 并通过 `signal_analyzer` 将全部功能部署为线上交互服务。前两组实验的输入均为长度约 14000 点的 `.mat` 信号, 采样率 $f_s = 4410$ Hz; 音乐实验的输入为 `self_songbie.mp3`, 按 44.1 kHz 读取处理。

3 给定信号的频谱特性分析

图 1 给出了两路信号的时域波形。可以看到, 二者的局部振荡方式和能量分布明显不同, 仅从波形外观就可以判断它们并非同一个信号。



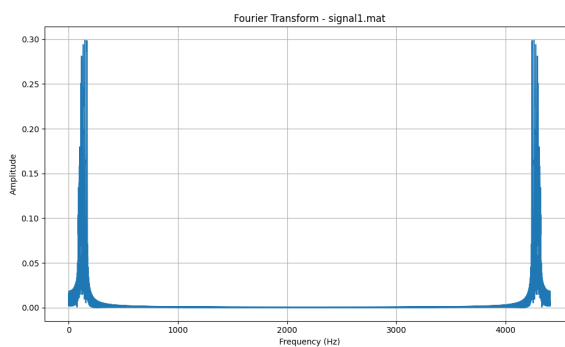
(c) 信号 1 时域波形



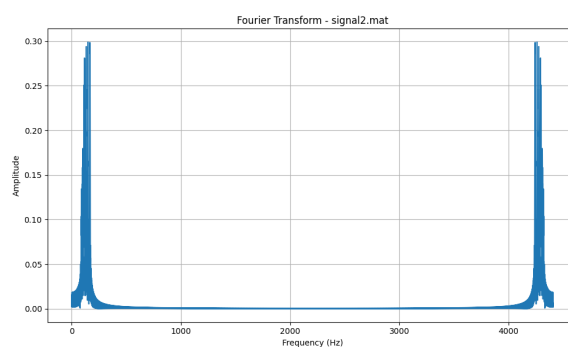
(d) 信号 2 时域波形

图 1: 给定信号的时域波形对比

随后利用 `py_tools/fft_analyze.py` 计算两路信号的频谱，结果如图 2 所示。



(a) 信号 1 频谱图



(b) 信号 2 频谱图

图 2: 给定信号的傅里叶变换结果

从图 2 可以得到结论：

两路信号的主峰位置与幅值几乎完全重合，典型峰值出现在 0-100Hz 和 4k-5kHz 附近，两个峰值关于 $\frac{f_s}{2}$ 对称分布。

进一步计算 (py_tools/signal_stats.py) 可知, 两路信号的均值、方差和总能量也基本一致。这说明它们包含了几乎相同的全局频率成分, 但这些频率成分在时间轴上的组织方式不同。仅凭普通傅里叶变换无法解释这种” 频谱相同、波形不同” 的现象, 因此必须引入时频分析。

表 1: 信号基本特征分析

Signal	Mean	Variance	Total Energy
signal1.mat	0.0049	1.0001	14001.6744
signal2.mat	0.0049	1.0001	14001.6744

4 短时傅里叶变换结果与窗长规律

4.1 实验设置

本节展示的 STFT 结果均采用矩形窗 (见 py_tools/stft_analyzer.py), 并令帧移为半个窗长 $frameshift = window_size/2$ 。为便于观察分辨率变化规律, 正文选取 $window_size = 700$ 、 1400 、 2800 多种代表性窗长进行比较, 相应参数如表 2 所示。

窗长 $window_size$	帧移 $frameshift$	频率分辨率 Δf	时间分辨率 Δt
700	350	6.30 Hz	79.4 ms
1400	700	3.15 Hz	158.7 ms
2800	1400	1.58 Hz	317.5 ms

表 2: 不同窗长下的 STFT 参数 ($f_s = 4410$ Hz)

4.2 信号 1 的时频图

下图给出了信号 1 在三种窗长下的 STFT 结果。可以看到, 信号 1 的主频随时间整体呈分段上升趋势: 较短窗长时, 频率变化发生的位置更容易被定位, 时频图中的边界更清晰; 较长窗长时, 频率条纹更细、更稳定, 但相邻时段之间的切换变得平滑。

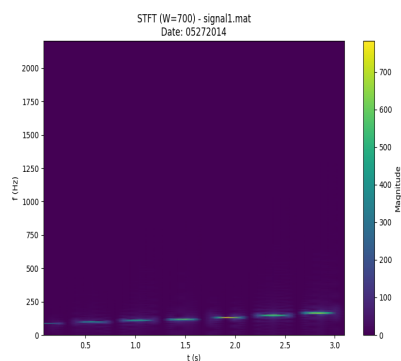


图 3: 信号 1 在窗长 700 下的时频图

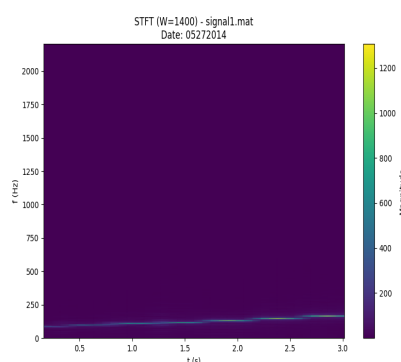


图 4: 信号 1 在窗长 1400 下的时频图

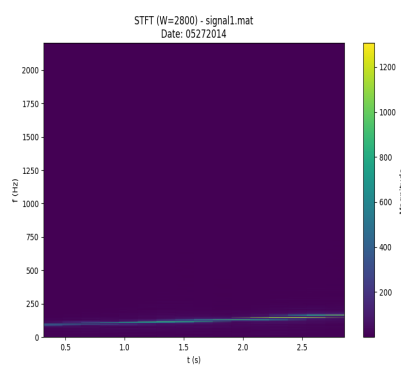


图 5: 信号 1 在窗长 2800 下的时频图

4.3 信号 2 的时频图

下图给出了信号 2 在相同参数下的 STFT 结果。与信号 1 相比，信号 2 仍然由相近的几个主频成分构成，但这些频率成分在时间上的排列顺序已经改变，局部能量分布也明显不同。

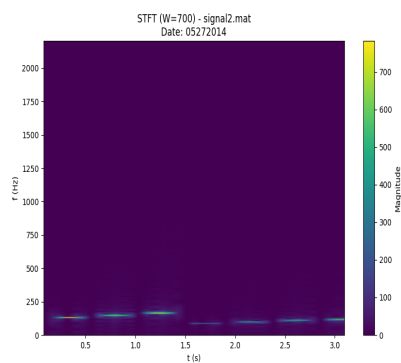


图 6: 信号 2 在窗长 700 下的时频图

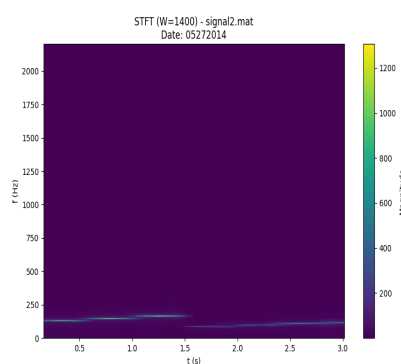


图 7: 信号 2 在窗长 1400 下的时频图

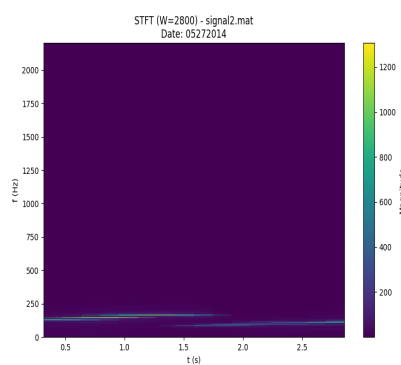


图 8: 信号 2 在窗长 2800 下的时频图

对比两组信号的时频图可以发现：

1. 两路信号的全局频率集合基本相同，这与频谱图的结论一致；
2. 信号 1 和信号 2 的主要频率成分的频率是逐步下降的，但是时间先后相互错位，对比 700 窗长的时频图非常明显。

3. 当窗长从 700 增大到 2800 时，频率条纹变得更细（频率分辨率提升），同时时间边界逐渐模糊（时间分辨率下降），与表 2 中的理论预测吻合。

因此，STFT 成功揭示了普通频谱图无法体现的”局部时序差异”，说明对于非平稳信号，仅有全局频谱远远不够，必须联合时间轴观察频率的出现过程。

5 音乐信号可视化分析

5.1 音乐信号处理流程

本节以 `self_songbie.mp3` 为输入，分别生成波形图、频谱图、时频图、滚动时频动画和彩虹雨频谱动画。三个脚本共享相同的参数配置：采样率 $f_s = 44100$ Hz，FFT 点数 $N_{\text{fft}} = 2^{15} = 32768$ ，帧移 $H = 2^{10} = 1024$ 。对应的分辨率为

$$\Delta f = \frac{f_s}{N_{\text{fft}}} = \frac{44100}{32768} \approx 1.35 \text{ Hz}, \quad \Delta t = \frac{H}{f_s} = \frac{1024}{44100} \approx 23.2 \text{ ms}.$$

此参数组合以牺牲极少的时间分辨率为代价，换取了极高的频率分辨率，适合展示音乐信号中细密的谐波结构。

三个脚本的具体处理流程如下：

1. **波形图(music_stft.py)**: 使用 `librosa.load` 读取音频并单声道化, 调用 `stft_analyzer.py` 中的 `stft` 函数得到复数时频矩阵，保留正频率部分计算幅值并转换为 dB 标度，在对数频率轴下绘制时频图与时域波形的组合快照；
2. **频谱图 (music_fft.py)**: 对整个音频信号做一次矩形窗 FFT，计算幅值谱并转换为 dB 标度，在对数频率轴下绘制频谱图；
3. **静态时频图 (music_stft.py)**: 使用 `librosa.load` 读取音频并单声道化，调用 `stft_analyzer.py` 中的 `stft` 函数得到复数时频矩阵，保留正频率部分计算幅值并转换为 dB 标度，在对数频率轴下绘制时频图与时域波形的组合快照；
4. **时频动画 (stft_animator.py)**: 在静态时频图基础上引入动态显示逻辑，每帧展示宽度为 10 帧的滑动时间窗口（约 $10 \times 23.2 \text{ ms} = 232 \text{ ms}$ ），使时频矩阵的局部切片随音乐播放连续滚动；时间戳以文字形式叠加在画面右上角；由 `FFMpegWriter` 先导出静音 MP4，再调用 `ffmpeg` 将原始音轨混流进最终视频；
5. **频谱动画 (fft_animator.py)**: 每帧对当前音频片段做矩形窗 FFT，将 dB 标度的正频率频谱绘制为折线（对数横轴），同时在背景中随机生成 120 条 HSV 彩色线段作为”彩虹雨”装饰，线段的横坐标在对数频率轴上均匀分布，纵坐标随机落在 dB 范围内，帧率与 STFT 动画一致，最终同样导出带音轨的 MP4。

5.2 静态分析

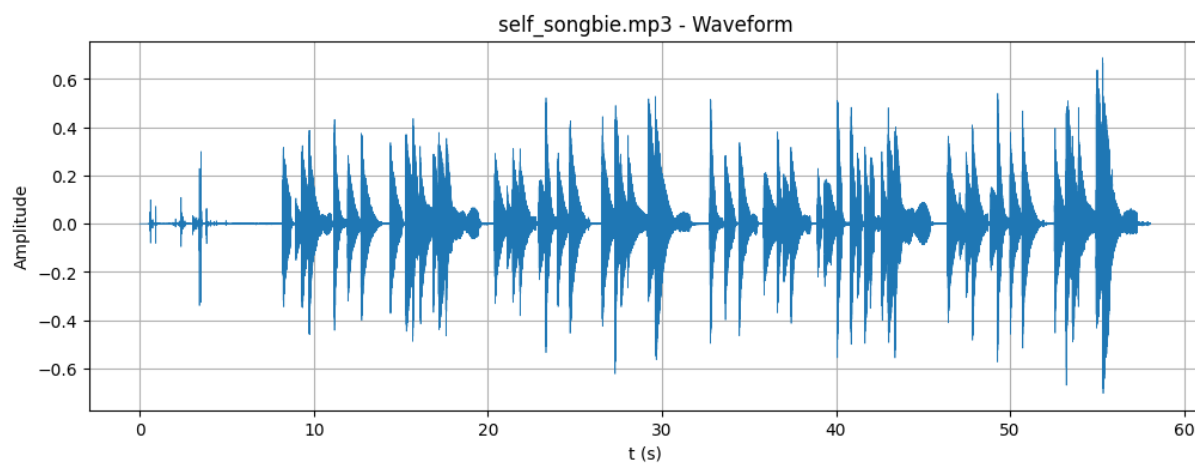


图 9: 音乐信号的波形图

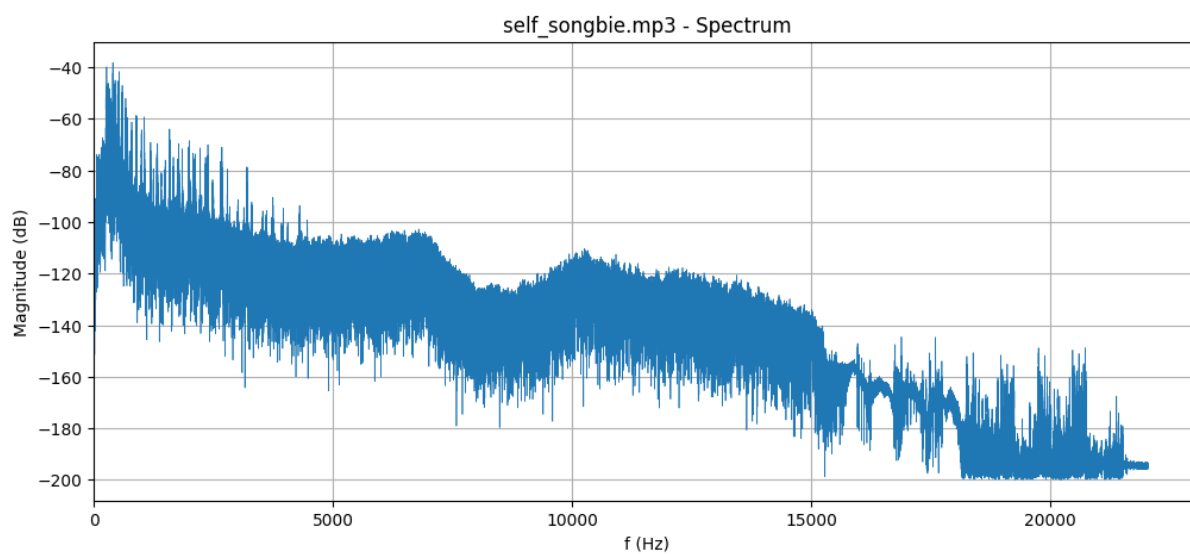


图 10: 音乐信号的频谱图

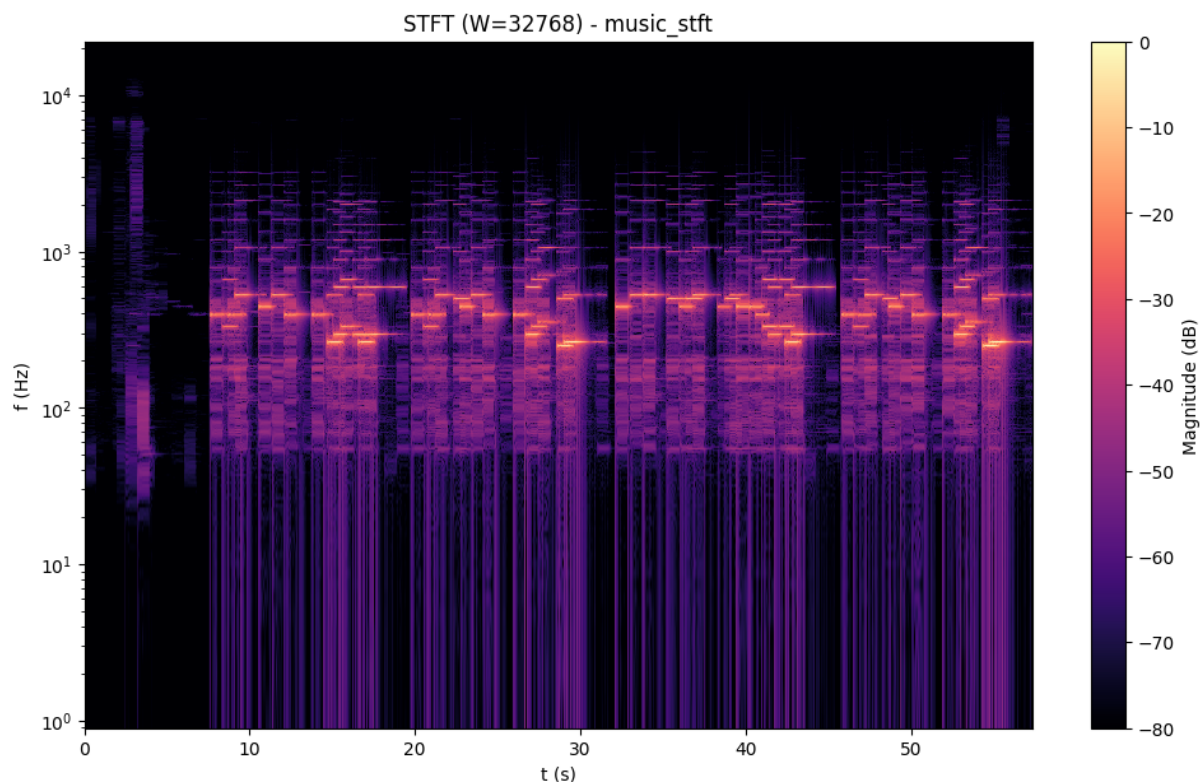


图 11: 音乐信号的时频图

从上图可以观察到以下特点：

1. 音乐信号的能量主要集中在中低频区域，主旋律与基频结构清晰可见；
2. 时频图中出现多组稳定的水平谐波纹理，反映了乐音的音高结构；
3. 在节拍或强起音附近，出现更宽的竖向能量扩展，对应打击感更强的瞬态成分。

5.3 时频动画分析

时频动画（见 `py_tools/stft_animator.py`）通过 10 帧滑动窗口实时展示时频矩阵的局部切片，使观察者能够随着音乐推进直观感受频率能量的时间演化。与静态时频图相比，动画形式能够将音高的起伏走向、和声叠加时频率条纹的聚合与分散、以及节拍处亮度的骤变展现得更加直观。由于每帧只显示约 232 ms 的时间窗口，局部频率结构的细节得以被放大，有助于分辨相邻音符之间的过渡。

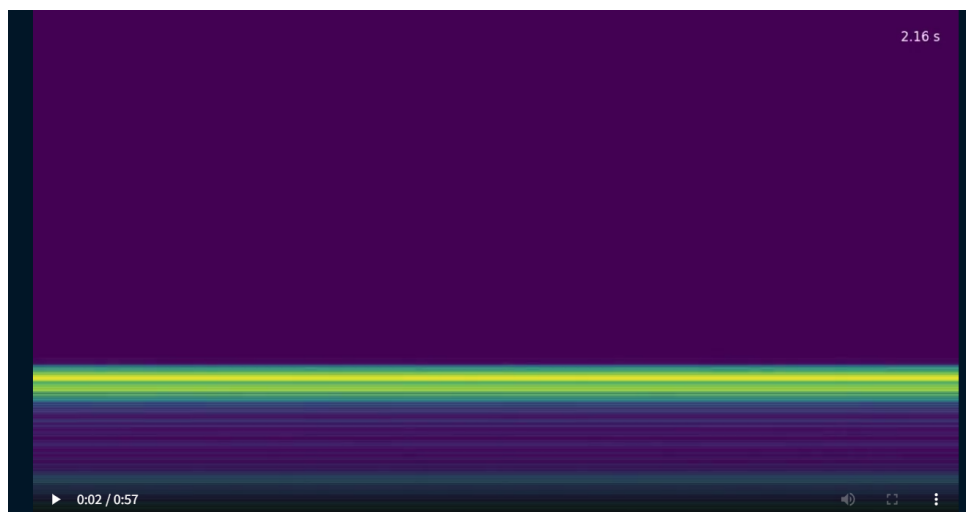


图 12: 时频动画截图

完整视频见附件 materials/music_stft.mp4

5.4 频谱动画分析

频谱动画（见 `py_tools/fft_animator.py`）以折线形式展示每一帧的瞬时 FFT 频谱，彩虹雨装饰背景使视觉效果更具感染力。由于每帧仅显示单帧 FFT 结果，与时频动画相比，频谱动画更直观地呈现了某一时刻各频率分量的相对强度；但缺少时间维度的二维展示，低频段谱线的起伏能够直接反映旋律的进行，高频段则可见伴奏和打击乐器的细节。

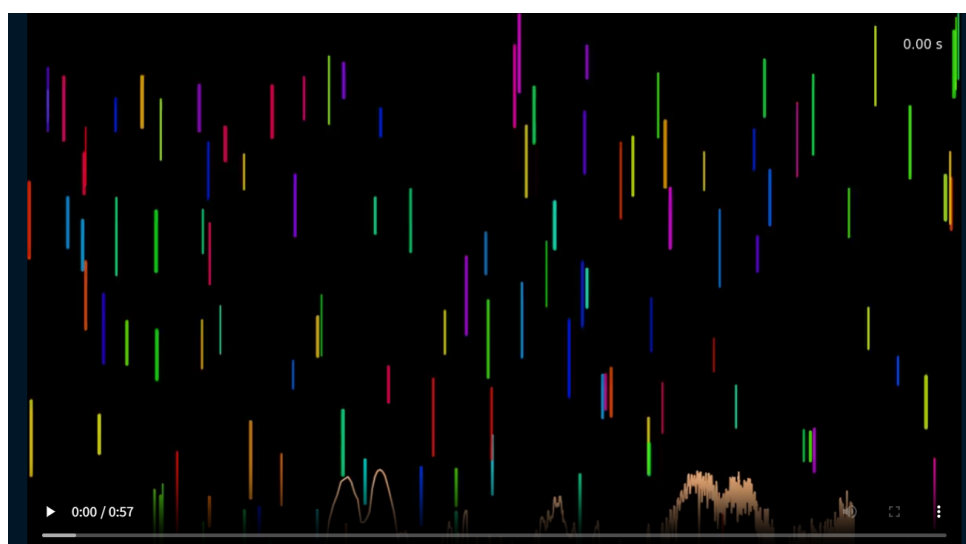


图 13: 频谱动画截图

完整视频见附件 materials/music_fft.mp4

5.5 Web 应用部署

为了方便他人复现和使用上述分析功能，本项目基于 `signal_analyzer` 模块构建了可交互网站，部署于魔搭社区（ModelScope）创空间，网址为：

<https://www.modelscope.cn/studios/XueZhe4/signal-analyzer>

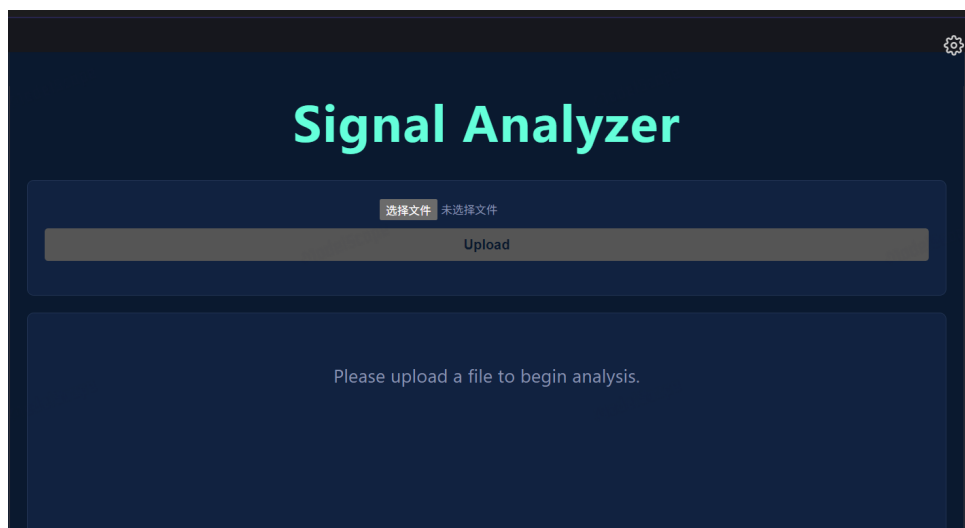


图 14: Web 应用界面截图

网站采用前后端分离架构，技术栈如下：

- **后端：**Python Flask，提供 REST API；核心接口包括 `POST /upload`（文件上传）、`GET /analyze/<filename>`（同步返回波形图、频谱图、时频图的 Base64 编码）和 `POST /animate/<filename>`（异步提交动画渲染任务，返回任务 ID）；
- **前端：**Vue 3 + TypeScript + Vite，提供参数输入表单（采样率、FFT 点数、帧移、窗函数类型、颜色方案等可调参数）、图像展示面板和视频播放器；
- **部署：**Docker Compose 编排前后端容器，Nginx 反向代理前端静态资源，FFmpeg 集成于 Docker 镜像以支持视频渲染。

与离线脚本相比，Web 后端的 STFT 实现在矩形窗之外增加了 Hann、Hamming、Blackman、Bartlett 和 Kaiser 五种窗函数，并支持独立设置 FFT 点数（零填充），为用户提供了更灵活的参数空间。动画渲染通过异步任务队列完成，避免长时间阻塞 HTTP 请求，前端可轮询任务状态后下载生成的 MP4 文件。

6 结论

本文围绕短时傅里叶变换的可视化展开了完整实验。首先，通过 `waveform.py` 和 `fft_analyze.py` 对两路给定信号进行时域观察与全局傅里叶变换，发现二者频谱几乎

一致，说明普通频谱只能描述“频率集合”，无法反映“频率出现顺序”；其次，通过 `stft_analyzer.py` 实现的矩形窗 STFT，对两路信号在不同窗长下的时频图进行了比较，验证了时间分辨率与频率分辨率之间的权衡关系，明确展示了两路信号在局部时序结构上的差异；再次，对自选音乐分别生成了滚动时频动画（`stft_animator.py`）和彩虹雨频谱动画（`fft_animator.py`），从不同角度展示了音频信号的动态频率结构；最后，将全部分析功能封装为 Flask 后端 API，结合 Vue.js 前端构建可交互网站并在线部署，使任意音频文件的时频分析可在浏览器中直接完成。

综上，STFT 是连接时域与频域的重要工具。对于非平稳信号，仅用傅里叶变换进行全局分析是不充分的；将滑动窗机制引入频谱计算后，才能获得更符合感知与工程需求的时间-频率联合表示。

A 核心代码

A.1 手写数值傅里叶变换（`lib/manual_fft.py`）

```

1 def fft(signal, time_samples, frequency_samples):
2     """按连续傅里叶变换的数值求和公式计算频谱。"""
3     values = np.asarray(signal, dtype=np.complex128).reshape(-1)
4     time_axis = np.asarray(time_samples, dtype=np.float64).reshape(-1)
5     freq_axis = np.asarray(frequency_samples, dtype=np.float64).reshape
6         (-1)
7     sample_interval = float(time_axis[1] - time_axis[0])
8     result = np.zeros(freq_axis.size, dtype=np.complex128)
9     for k in range(freq_axis.size):
10         for i in range(time_axis.size):
11             result[k] += (
12                 values[i]
13                 * np.exp(-2j * np.pi * freq_axis[k] * time_axis[i])
14                 * sample_interval
15             )
16     return result

```

A.2 STFT 核心实现（`stft_analyzer.py`）

```

1 def stft(signal, fs, window_size, frame_shift=None):
2     """矩形窗 STFT，默认半窗重叠，逐帧调用 numpy FFT。"""
3     samples = np.asarray(signal, dtype=np.float64).reshape(-1)

```

```

4     if frame_shift is None:
5         frame_shift = max(window_size // 2, 1)
6     nfft = window_size
7     num_frames = (samples.size - window_size) // frame_shift + 1
8     window = np.ones(window_size, dtype=np.float64) # 矩形窗
9     spectrum = np.zeros((nfft, num_frames), dtype=np.complex128)
10    frequencies = np.arange(nfft, dtype=np.float64) * fs / nfft
11    for m in range(num_frames):
12        start = m * frame_shift
13        frame = samples[start:start + window_size] * window
14        spectrum[:, m] = np.fft.fft(frame, n=nfft)
15    times = (np.arange(num_frames) * frame_shift + window_size / 2) / fs
16    return frequencies, times, spectrum

```

A.3 彩虹雨效果 (fft_animator.py)

```

1  def _build_rainbow_rain_segments(rng, min_frequency, max_frequency,
2                                  min_db, max_db):
3      """每帧生成120条对数均匀分布的HSV彩色线段作为装饰背景。"""
4      drop_count = Config.rain_drop_count # 120
5      log_x = rng.uniform(
6          np.log10(min_frequency), np.log10(max_frequency), drop_count
7      )
8      x_positions = np.power(10.0, log_x)
9      y_starts = rng.uniform(min_db, max_db, drop_count)
10     lengths = rng.uniform(
11         Config.rain_length_min, Config.rain_length_max, drop_count
12     )
13     y_ends = np.maximum(y_starts - lengths, min_db)
14     colors = plt.cm.hsv(rng.random(drop_count))
15     widths = rng.uniform(
16         Config.rain_width_min, Config.rain_width_max, drop_count
17     )
18     segments = np.stack((
19         np.column_stack((x_positions, y_starts)),
20         np.column_stack((x_positions, y_ends)),
21     ), axis=1)
22     return segments, colors, widths

```

